Ministerul Educaţiei şi Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică şi Microelectronică

Departamentul Ingineria Software şi Automatică

Report on Laboratory Work No. 7

Processes, Redirection, and Pipes

Operating Systems

Done by: st. gr. FAF-241, Caţ Chiril

Checked by: asist. univ. Reiţman Patricia

Chişinău – 2026

Contents

1 Purpose and environment **1**

2 Fork, exec, and wait **1**

3 A zombie before waitpid **2**

4 Redirecting standard output **2**

5 Connecting commands with a pipe **3**

6 Handout questions and answers **4**

6.1 Part 1: fork, exec, and wait 4

6.2 Part 2: a zombie 4

6.3 Part 3: redirection 4

6.4 Part 4: a pipe 5

7 Conclusion **5**

1 Purpose and environment

I built and ran four POSIX C programs to observe process creation, program replacement, child reaping, standard-output redirection, and a two-stage pipe. The experiments ran on macOS on 26 October 2026. The `gcc` command reported Apple clang 21.0.0. I compiled each source with `-std=c11 -Wall -Wextra -Werror` and used a temporary directory for binaries and generated files. The submitted `code/` directory contains only the four required C sources. A second source copy is stored in `xv6-riscv/lab7-linux/`; the programs use POSIX APIs and are outside xv6's `UPROGS`. The required `report.md` contains the full Observe answers and selected command outputs.

2 Fork, exec, and wait

The final `run.c` forks, executes `ls -l` in the child, and waits in the parent. For the two requested variations, I compiled temporary copies that replaced the `execlp` line with `date` and `nosuchprog`; the submitted source remains the original `ls` version.

```
$ ./run
parent: my PID is 55082
child:  my PID is 55084, my parent is 55082
total 536
... (directory listing)
parent: child 55084 ended with status 0
$ ./run_date
parent: my PID is 52683
child:  my PID is 52695, my parent is 52683
Mon Oct 26 11:05:35 EET 2026
parent: child 52695 ended with status 0
$ ./run_missing
parent: my PID is 52681
child:  my PID is 52694, my parent is 52681
exec: No such file or directory
parent: child 52694 ended with status 127
```

`perror("exec")` runs only for `nosuchprog` because a successful `execlp` replaces the child's process image and never returns. The child's parent value is the original process's PID: 55082 in the `ls` run. The parent observes status 0 for `ls` and 127 for the missing program, matching the child's explicit `exit(127)` after the error.

3 A zombie before waitpid

`zombie.c` is the original `ls` program with `sleep(20)` inserted before `waitpid`. After two seconds, the parent was sleeping and its child had exited but had not yet been reaped:

```
$ ./zombie > /dev/null &
$ sleep 2
$ ps -o pid,ppid,stat,command -p 54291
  PID  PPID STAT COMMAND
54291 54289 SN   ./zombie
$ ps -axo pid,ppid,stat,command | awk '$2 == 54291 {print}'
54293 54291 ZN   <defunct>
$ wait
$ ps -axo pid,ppid,stat,command | awk '$2 == 54291 {print}'
(no rows)
```

The `Z` in `STAT` means zombie. The additional `N` is a macOS scheduling flag. After 20 seconds the defunct child disappeared because the parent's `waitpid(pid, &status, 0)` collected its exit status.

4 Redirecting standard output

The child in `redirect.c` closes descriptor 1, opens `out.txt`, then executes `ls -l`. The new file gets the lowest available number:

```
$ ./redirect
open gave descriptor 1
done: the listing is in out.txt
$ head -3 out.txt
total 448
-rw-r--r--@ 1 chirilcat  wheel      0 Oct 26 11:05 out.txt
-rwxr-xr-x@ 1 chirilcat  wheel  33848 Oct 26 11:05 pipeline
```

`open` returned descriptor 1 because it was just closed. The unchanged `ls` writes to descriptor 1, which now refers to the file, so its listing is absent from the screen. The diagnostic still goes to descriptor 2 (standard error). Redirecting errors would require closing descriptor 2 before opening the error file, or assigning the file to 2 with `dup2`.

5 Connecting commands with a pipe

`pipeline.c` uses a pipe and two forks. The `ls` child maps the pipe's writing end to descriptor 1; the `wc` child maps its reading end to descriptor 0. Each process closes pipe descriptors it no longer uses.

```
$ ./pipeline
      77
$ ls /etc | wc -l
      77
```

The C program and the shell pipeline produced the same count. I then built a temporary version with the parent's `close(p[1]);` commented out. After two seconds, `ls` had exited while `wc` and the parent were still waiting:

```
  PID  PPID STAT COMMAND
54803 54801 SN   ./pipeline_broken
54806 54803 SN   wc -l
```

`wc` cannot observe end-of-file while the parent still holds a copy of the pipe's write end. I stopped the temporary processes after recording their state. The submitted `pipeline.c` closes both pipe ends in the parent.

6 Handout questions and answers

Every required Observe question from the Lab 7 handout is stated here with its answer. The measurements and process-table output are shown in the preceding sections.

6.1 Part 1: fork, exec, and wait

1. **Question:** The line `perror("exec")` is in the child, right after `execlp`. Why is it printed only with `nosuchprog`?

   Answer: Successful `execlp` replaces the child program and never returns to that line. The lookup of `nosuchprog` fails, so `execlp` returns and `perror` prints the error.

2. **Question:** Which number does the child print as "my parent", and whose PID is it?

   Answer: In the recorded `ls` run it printed 55082, the PID printed by the original parent process. The particular PID changes from run to run.

3. **Question:** What status does the parent see with `ls`, and what status with `nosuchprog`?

   Answer: It saw 0 for successful `ls` and 127 for `nosuchprog`. The child explicitly called `exit(127)` after the failed `execlp`.

6.2 Part 2: a zombie

1. **Question:** Which letter does `ls` have in the `STAT` column, and what does it mean?

   Answer: Its `STAT` began with `Z`, meaning zombie: the child had exited, but the parent had not yet collected its status. The host also displayed `N`, a scheduling flag.

2. **Question:** After 20 seconds, is the `[ls] <defunct>` line still there? Which line of `zombie.c` made it disappear?

   Answer: No. The process-table check after the parent finished found no child row. The `waitpid(pid, &status, 0)` line reaped the child after `sleep(20)`.

6.3 Part 3: redirection

1. **Question:** Which number did `open` give, and why exactly that one?

   Answer: It returned descriptor 1 because the child had closed standard output, leaving 1 as the lowest free descriptor.

2. **Question:** `ls` was not changed at all. Why does its listing not appear on the screen?

   Answer: `ls` writes to descriptor 1 as usual. The child made descriptor 1 refer to `out.txt` before `exec`, so the listing went to the file.

3. **Question:** To redirect the errors instead (like `2> err.txt`), which descriptor would you close before `open`?

   Answer: Close descriptor 2, standard error. The next `open` would then reuse 2, assuming no lower descriptor is free.

6.4 Part 4: a pipe

1. **Question:** Did `./pipeline` and `ls /etc | wc -l` give the same number?

   Answer: Yes. Both printed 77 on the measured host.

2. **Question:** After the change, which of the two children never ends, `ls` or `wc`?

   Answer: `wc` remained waiting. The process table showed `wc -l` still active after `ls` had exited.

3. **Question:** Why, in one sentence: what is `wc` waiting for?

   Answer: `wc` is waiting for end-of-file on the pipe, which cannot arrive while the parent still holds its write end open.

7 Conclusion

`fork` creates a child, `exec` replaces its program, and `waitpid` collects its exit status. Redirection and pipelines assign files or pipe endpoints to standard descriptors before `exec`. The broken-pipe experiment showed why every unused write end must be closed so the reader can receive end-of-file.